

Assignment 1 — Answer Reference (Structs / Arrays / Linked List)

Note on the given code: the linked list section has a typo — `Struct node* lnk;` should be `struct node* link;` (lowercase struct, and link not lnk). I've used the corrected version below. If your professor's original handout also has this typo, just copy it faithfully unless told otherwise.

Assumptions I made (flag these to your prof if graded strictly, or adjust to match class conventions): - shelves[0] = top shelf, shelves[1] = middle shelf, shelves[2] = bottom shelf - "Last book" on a shelf = the last **occupied** slot, i.e. index bookCount - 1 (not index 19, which may be empty) - BointR points to BS (i.e. BointR = &BS;) - Array question 1 checks uniqueness by bookName - Linked list question 1 inserts in alphabetical order by bookName (the prompt doesn't define "proper position," so state your own assumption on your paper too — this is the most common interpretation for this kind of exercise)

STRUCTURES

Reference recap:

```
#define MAX 30
typedef char String[50];

typedef struct{
    String bookName;
    String authorName;
    String publishingHouse;
    int publishingYear;
    int totalPageNum;
}Book;

typedef struct{
    Book books[20];
    int bookCount;
}Shelf;

typedef struct{
    Shelf shelves[3];
    int totalBookCount;
}BookShelf;

BookShelf BS;
BookShelf *BointR; // assume: BointR = &BS;
```

1. Display the bookName of the 3rd book of the top shelf

```
// w/o pointer
printf("%s", BS.shelves[0].books[2].bookName);

// w/ pointer
printf("%s", BointR->shelves[0].books[2].bookName);
```

2. Update the publishing year of the last book of the bottom shelf

```
// w/o pointer
BS.shelves[2].books[BS.shelves[2].bookCount - 1].publishingYear = 2024;

// w/ pointer
BointR->shelves[2].books[BointR->shelves[2].bookCount - 1].publishingYear = 2024;
```

3. Display how many books there are on the top and middle shelf

```
// w/o pointer
printf("%d", BS.shelves[0].bookCount + BS.shelves[1].bookCount);

// w/ pointer
printf("%d", BointR->shelves[0].bookCount + BointR->shelves[1].bookCount);
```

4. Access the 7th letter of the authorName of the 6th book of the bottom shelf (6th book = index 5, 7th letter = index 6)

```
// w/o pointer
printf("%c", BS.shelves[2].books[5].authorName[6]);

// w/ pointer
printf("%c", BointR->shelves[2].books[5].authorName[6]);
```

5. Update the 2nd book on the middle shelf — new copy, 400 total pages

```
// w/o pointer
BS.shelves[1].books[1].totalPageNum = 400;

// w/ pointer
BointR->shelves[1].books[1].totalPageNum = 400;
```

6. Display the 1st letter of the publishingHouse of the last book on the top shelf

```
// w/o pointer
printf("%c", BS.shelves[0].books[BS.shelves[0].bookCount - 1].publishingHouse[0]);

// w/ pointer
printf("%c", BointR->shelves[0].books[BointR->shelves[0].bookCount - 1].publishingHouse[0]);
```

7. The 7th book on the bottom shelf's author should be "J.R.R. Tolkein" (index 6)

```
// w/o pointer
strcpy(BS.shelves[2].books[6].authorName, "J.R.R. Tolkein");

// w/ pointer
strcpy(BointR->shelves[2].books[6].authorName, "J.R.R. Tolkein");
```

8. Total book count of the entire bookshelf, currently empty — how do you fill it? Sum the bookCount of all three shelves and assign it:

```
// w/o pointer
BS.totalBookCount = BS.shelves[0].bookCount + BS.shelves[1].bookCount + BS.shelves[2].bookCount;

// w/ pointer
BointR->totalBookCount = BointR->shelves[0].bookCount + BointR->shelves[1].bookCount + BointR->shelves[2].bookCount;
```

9. Insert at the last position, a book "Moby" into the middle shelf

```
// w/o pointer
strcpy(BS.shelves[1].books[BS.shelves[1].bookCount].bookName, "Moby");
BS.shelves[1].bookCount++;

// w/ pointer
strcpy(BointR->shelves[1].books[BointR->shelves[1].bookCount].bookName, "Moby");
BointR->shelves[1].bookCount++;
```

10. Interchange the first and last book of the top shelf

```
// w/o pointer
Book temp = BS.shelves[0].books[0];
BS.shelves[0].books[0] = BS.shelves[0].books[BS.shelves[0].bookCount - 1];
BS.shelves[0].books[BS.shelves[0].bookCount - 1] = temp;

// w/ pointer
Book temp2 = BointR->shelves[0].books[0];
BointR->shelves[0].books[0] = BointR->shelves[0].books[BointR->shelves[0].bookCount - 1];
BointR->shelves[0].books[BointR->shelves[0].bookCount - 1] = temp2;
```

ARRAYS

1. Return 0 if books in top shelf are not all unique, 1 if they are

a. Function Header

```
int checkUnique(BookShelf BS);
```

b. Function Call

```
int result = checkUnique(BS);
```

c. Code

```
int checkUnique(BookShelf BS){
    int i, j;
    int count = BS.shelves[0].bookCount;

    for(i = 0; i < count - 1; i++){
        for(j = i + 1; j < count; j++){
            if(strcmp(BS.shelves[0].books[i].bookName, BS.shelves[0].books[j].bookName) == 0)
                return 0; // duplicate found
        }
    }
    return 1; // all unique
}
```

d. Simulation Suppose top shelf has bookCount = 3 with bookNames: "Dune", "1984", "Dune".

i	j	Compare	Match?	Action
0	1	"Dune" vs "1984"	No	continue
0	2	"Dune" vs "Dune"	Yes	return 0

Function exits early, returns 0 (not unique). If instead the names were "Dune", "1984", "Sapiens", every comparison would fail to match, the loops finish, and the function returns 1.

2. Delete all occurrences of a specified publishingHouse across the whole BookShelf

a. Function Header

```
void deleteByPublisher(BookShelf *Bointer, String w);
```

b. Function Call

```
deleteByPublisher(BointR, "Penguin Random House");
```

c. Code

```
void deleteByPublisher(BookShelf *Bointer, String w){
    int s, i, j;

    for(s = 0; s < 3; s++){
        for(i = 0; i < Bointer->shelves[s].bookCount; i++){
            if(strcmp(Bointer->shelves[s].books[i].publishingHouse, w) == 0){
                // shift every book after i one slot to the left
                for(j = i; j < Bointer->shelves[s].bookCount - 1; j++){
                    Bointer->shelves[s].books[j] = Bointer->shelves[s].books[j + 1];
                }
                Bointer->shelves[s].bookCount--;
                i--; // re-check this index; a new book shifted into it
            }
        }
    }
}
```

d. Simulation Suppose the top shelf (bookCount = 3) has publishingHouse values: "Penguin", "Scholastic", "Penguin", and w = "Penguin".

i	books[i].publishingHouse	Match?	Action	bookCount after
0	"Penguin"	Yes	shift books[1],[2] left → books = {Scholastic, Penguin}; i--	2
0 (recheck)	"Scholastic"	No	continue	2
1	"Penguin"	Yes	shift (nothing after it) → books = {Scholas- tic}; i--	1

i	books[i].publishingHouse	Match?	Action	bookCount after
0 (recheck)	"Scholastic"	No	loop ends (i=1 not < book- Count=1)	1

Result: only "Scholastic" remains on the top shelf. Repeat the same trace for the middle and bottom shelves.

LINKED LIST

Reference recap (typo corrected):

```
#define MAX 30
typedef char String[50];

typedef struct{
    String bookName;
    String authorName;
    String publishingHouse;
    int publishingYear;
    int totalPageNum;
}Books;

typedef struct node{
    Books book;
    struct node* link;    // corrected from "Struct node* lnik"
}Shelf;

typedef struct{
    Shelf *BookNook;
    int totalBookCount;
}BookShelf;
```

1. Insert an array of 5 books into BSN, each in its proper (alphabetical, by bookName) position

a. Function Header

```
void insertSorted(BookShelf *BSN, Books arr[], int size);
```

b. Function Call

```
insertSorted(&BSN, bookArray, 5);
```

c. Code

```
void insertSorted(BookShelf *BSN, Books arr[], int size){
    int i;

    for(i = 0; i < size; i++){
        Shelf *newNode = (Shelf*)malloc(sizeof(Shelf));
        newNode->book = arr[i];
        newNode->link = NULL;
```

```

    if(BSN->BookNook == NULL || strcmp(arr[i].bookName, BSN->BookNook->book.bookName) < 0)
        // insert at head
        newNode->link = BSN->BookNook;
        BSN->BookNook = newNode;
    } else {
        // find the correct spot
        Shelf *cur = BSN->BookNook;
        while(cur->link != NULL && strcmp(arr[i].bookName, cur->link->book.bookName) > 0)
            cur = cur->link;
        }
        newNode->link = cur->link;
        cur->link = newNode;
    }
    BSN->totalBookCount++;
}
}

```

d. Simulation Say `BSN->BookNook = NULL` and the array (size 5) has bookNames in this order: "Emma", "Dune", "1984", "Fable", "Beloved".

Insert	List before	Comparison	List after
"Emma"	(empty)	head is NULL → insert at head	Emma
"Dune"	Emma	"Dune" < "Emma" → insert at head	Dune → Emma
"1984"	Dune → Emma	"1984" < "Dune" → insert at head	1984 → Dune → Emma
"Fable"	1984 → Dune → Emma	"Fable" > "1984", > "Dune", < "Emma" → insert between Dune and Emma	1984 → Dune → Fable → Emma
"Beloved"	1984 → Dune → Fable → Emma	"Beloved" > "1984", < "Dune" → insert between them	1984 → Beloved → Dune → Fable → Emma

Final list: **1984 → Beloved → Dune → Fable → Emma**, `totalBookCount = 5`.

(Execution-stack style: each call to `insertSorted` runs once per element of `arr`, walking `cur` from `BSN->BookNook` until it finds the insertion point, then relinking two pointers — `newNode->link` and either `BSN->BookNook` or `cur->link`.)